

Thiết kế và tối ưu hàm băm

Li Yuxiu

Trường Trung học Nankai, Thiên Tân

2005

Nguồn gốc: Thiết kế và tối ưu hàm băm

IOI China candidate team papers / 2005 / Li Yuxiu.doc

Tóm tắt

Băm là một cấu trúc dữ liệu thường dùng trong thi tin học. Một hàm băm tốt có thể cải thiện đáng kể hiệu quả thời gian và bộ nhớ của toàn bộ chương trình. Bài viết thảo luận cách thiết kế hàm băm cho nhiều loại mẫu khác nhau, tức các giá trị khóa, đồng thời phân tích và tổng kết một số hàm băm thường dùng.

Từ khóa: băm; hàm băm; chuỗi; số nguyên; số thực; hoán vị và tổ hợp.

1 Tiêu chí đánh giá

Với một hàm băm, tiêu chí quan trọng để đánh giá tốt hay xấu là tính ngẫu nhiên: với một tập mẫu bất kỳ, xác suất rơi vào từng ô của bảng băm có đều hay không. Xác suất càng đều thì dữ liệu phân bố trong bảng càng đều và hiệu suất dùng bộ nhớ càng cao. Trong thi đấu, tính chất của mẫu thường không thể biết trước, nên suy luận toán học bị hạn chế. Bài viết dùng thực nghiệm để quan sát tính ngẫu nhiên ấy.

2 Hàm băm cho số nguyên

Ba phương pháp thường gặp là lấy dư trực tiếp, nhân rồi lấy phần nguyên, và lấy giữa bình phương. Giả sử khóa là k , dung lượng bảng băm là m , và hàm băm là $h(k)$.

2.1 Lấy dư trực tiếp

Ta chia khóa k cho m và dùng phần dư làm vị trí trong bảng:

$$h(k) = k \bmod m.$$

Ví dụ, nếu dung lượng bảng là 701 và khóa là 1234, vị trí là $1234 \bmod 701 = 533$.

Ưu điểm của phương pháp này là dễ cài đặt, chạy nhanh, nên rất thường dùng. Nhược điểm là nếu m chọn không tốt và mẫu lại có cấu trúc đặc biệt, dữ liệu dễ dồn vào một số ô. Theo kinh nghiệm, m thường nên là một số nguyên tố không quá gần một lũy thừa đặc biệt; nếu miền giá trị của khóa nhỏ, có thể chọn m trong khoảng 1.1 tới 1.6 lần miền giá trị. Khi thi đấu, có thể viết bộ sinh số nguyên tố hoặc chọn một số “trông giống nguyên tố”.

Tác giả thử chèn 4000 số vào bảng dung lượng 701:

Dữ liệu	ngẫu nhiên	liên tiếp
ô nhỏ nhất	0	5
ô lớn nhất	15	6
kỳ vọng	5.70613	5.70613
độ lệch chuẩn	2.4165	0.455531

Với dữ liệu ngẫu nhiên, ô lớn nhất gần gấp ba kỳ vọng. Trên máy Pentium III 866MHz, 128MB RAM của tác giả, một lần tính mất khoảng 39 ns, tức khoảng 35 chu kỳ.

2.2 Nhân rồi lấy phần nguyên

Chọn một số thực $A \in (0, 1)$, tốt nhất là vô tỉ. Tính kA , lấy phần thập phân, nhân với m , rồi lấy phần nguyên:

$$h(k) = \lfloor m\{kA\} \rfloor.$$

Ở đây $\{x\} = x - \lfloor x \rfloor$. Ví dụ có thể chọn $A = (\sqrt{5} - 1)/2$, một lựa chọn thường cho hiệu quả tốt.

Với 4000 số và bảng dung lượng 701, tác giả ghi nhận:

Dữ liệu	ngẫu nhiên	liên tiếp
ô nhỏ nhất	0	4
ô lớn nhất	15	7
kỳ vọng	5.70613	5.70613
độ lệch chuẩn	2.5069	0.619999

Từ công thức có thể thấy phương pháp này ít phụ thuộc vào m , nên khi m không phù hợp cho lấy dư trực tiếp, nó có thể biểu hiện tốt. Tuy nhiên trong thử nghiệm trên, kết quả không quá lý tưởng; hơn nữa do dùng nhiều phép toán dấu phẩy động, tốc độ chậm. Sau nhiều lần tối ưu, tác giả vẫn cần khoảng 892 ns cho một lần tính, tức khoảng 810 chu kỳ, chậm hơn lấy dư trực tiếp khoảng 23 lần.

2.3 Lấy giữa bình phương

Ta bình phương khóa k , rồi lấy một số bit hoặc chữ số ở giữa làm giá trị băm. Vì mỗi chữ số của k đều ảnh hưởng tới phần giữa của k^2 , phương pháp này thường có hiệu quả khá tốt. Tuy nhiên với k nhỏ, hiệu quả không lý tưởng; cài đặt cũng rườm rà hơn. Để tận dụng không gian bảng băm, dung lượng m nên là lũy thừa của 2.

Với 4000 số và bảng dung lượng 512, tác giả ghi nhận:

Dữ liệu	ngẫu nhiên	liên tiếp
ô nhỏ nhất	0	1
ô lớn nhất	17	17
kỳ vọng	7.8125	7.8125
độ lệch chuẩn	2.95804	2.64501

Kết quả kém hơn tưởng tượng, nhất là với dữ liệu liên tiếp. Nhưng do chỉ dùng phép nhân và phép bit, nó rất nhanh: khoảng 23 ns, tức 19 chu kỳ, còn nhanh hơn lấy dư trực tiếp.

2.4 So sánh

Phương pháp	Cài đặt	Hiệu quả	Tốc độ
lấy dư trực tiếp	dễ	tốt	khá nhanh
nhân lấy phần nguyên	khá dễ	khá tốt	chậm
lấy giữa bình phương	trung bình	khá tốt	nhanh

Từ bảng này có thể thấy lấy dư trực tiếp có tỷ lệ hiệu quả trên độ phức tạp tốt nhất, nên cũng là phương pháp thường dùng nhất trong thi đấu.

Với số thực, có thể trực tiếp dùng phương pháp nhân rồi lấy phần nguyên. Với những loại dữ liệu khác, ta thường chuyển chúng thành số nguyên trước rồi mới đưa vào bảng băm.

3 Hàm băm cho chuỗi

Bản thân một chuỗi có thể xem là một số nguyên lớn trong cơ số 256; với chuỗi ANSI có thể xem là cơ số 128. Vì vậy có thể dùng lấy dư trực tiếp và tính giá trị trong thời gian tuyến tính theo độ dài chuỗi. Để đảm bảo hiệu quả, m vẫn không nên chọn quá gần các giá trị đặc biệt. Đặc biệt, nếu xem chuỗi là số trong cơ số b , chọn $m = b - 1$ sẽ làm mọi hoán vị của cùng một chuỗi có cùng giá trị băm, vì giá trị theo modulo $b - 1$ chỉ phụ thuộc vào tổng chữ số.

Các hàm băm chuỗi thường dùng còn có ELFHash, APHash, v.v. Chúng dùng phép toán bit để làm mỗi ký tự ảnh hưởng tới giá trị cuối. Ngoài ra còn có các hàm băm mật mã như MD5 và SHA1, vốn rất khó tìm và chậm trong bối cảnh thông thường.

Tác giả lấy ngẫu nhiên 1000 từ khác nhau và 1000 câu khác nhau từ một tiểu thuyết của Mark Twain, lần lượt làm dữ liệu chuỗi ngắn và chuỗi dài. Sau khi dùng các hàm băm biến chúng thành số nguyên, lại lấy dư trực tiếp vào bảng dung lượng 1237; nếu xung đột thì chuỗi mới ghi đè chuỗi cũ. Số ô còn được phủ sau cùng phản ánh thô hiệu quả của hàm băm:

Hàm	ngắn	dài	trung bình
lấy dư trực tiếp	667	676	671.5
P. J. Weinberger	683	676	679.5
ELF	683	676	679.5
SDBM	694	680	687.0
BKDR	665	710	687.5
DJB	694	683	688.5
AP	684	698	691.0
RS	691	693	692.0
JS	684	708	696.0

Đưa 1000 số ngẫu nhiên vào bảng dung lượng 1237 bằng lấy dư trực tiếp cũng chỉ phủ được 694 ô, nên vài phương pháp phía sau đã gần tới giới hạn ngẫu nhiên. Tác giả thường chọn JS Hash hoặc SDBM Hash. Hai đoạn mã được trích lại nguyên văn vì chúng là mã C chuẩn, không phải nội dung cần dịch:

```
unsigned int JSHash(char *str)
{
    unsigned int hash = 1315423911;
    while (*str)
        hash ^= ((hash << 5) + (*str++) + (hash >> 2));
    return (hash & 0x7FFFFFFF);
}
```

```

unsigned int SDBMHash(char *str)
{
    unsigned int hash = 0;
    while (*str)
        hash = (*str++) + (hash << 6) + (hash << 16) - hash;
    return (hash & 0x7FFFFFFF);
}

```

JSHash có phép tính phức tạp hơn; nếu yêu cầu hiệu quả không quá khắt khe, SDBMHash là một lựa chọn tốt.

4 Hàm băm cho hoán vị

Chính xác hơn, phần này không chỉ là “hashing”, mà là một quá trình đánh số (*numerize*): xây dựng song ánh giữa các hoán vị và các số tự nhiên. Khi đó ta có thể định địa chỉ trực tiếp, dùng làm hàm băm, hoặc dùng trong quy hoạch động nén trạng thái.

4.1 Kiến thức nền

Ta quen với biểu diễn cơ số b :

$$x = a_0 + a_1b + a_2b^2 + \dots.$$

Tương tự, dùng hệ cơ số biến đổi dựa trên giai thừa. Từ đẳng thức

$$n! = (n-1)(n-1)! + (n-1)!,$$

có thể suy ra mọi số nguyên từ 0 tới $n! - 1$ đều biểu diễn duy nhất được dưới dạng

$$x = a_1 \cdot 1! + a_2 \cdot 2! + \dots + a_{n-1} \cdot (n-1)!,$$

với

$$0 \leq a_i \leq i.$$

Có thể hiểu dãy $a_{n-1}, \dots, a_1$ là một “số cơ số biến đổi”: chữ số thứ nhất có cơ số 2, chữ số thứ hai có cơ số 3, v.v. Vì có đúng $n!$ dãy như vậy, ta tự nhiên liên hệ chúng với $n!$ hoán vị của n phần tử.

4.2 Song ánh giữa hoán vị đầy đủ và số tự nhiên

Giả sử các phần tử là $1, 2, \dots, n$. Với một hoán vị p , đặt a_i là số phần tử nhỏ hơn i nằm bên phải i trong hoán vị. Dãy các a_i chính là mã Lehmer theo giá trị phần tử.

Ví dụ, hoán vị 4213 của 1, 2, 3, 4 có: bên phải 4 có 3 số nhỏ hơn, bên phải 3 có 0 số nhỏ hơn, bên phải 2 có 1 số nhỏ hơn. Vì vậy mã là 301, ứng với

$$3 \cdot 3! + 0 \cdot 2! + 1 \cdot 1! = 19.$$

Ngược lại, từ 19 có thể khôi phục mã 301, rồi khôi phục hoán vị 4213.

4.3 Hoán vị chọn một phần

Ta cũng có thể đánh số các chỉnh hợp chọn n phần tử từ m phần tử. Theo nguyên lý nhân,

$$P(m, n) = m(m - 1)(m - 2) \cdots (m - n + 1).$$

Ở vị trí thứ nhất có m cách chọn, vị trí thứ hai có $m - 1$ cách, v.v. Vì vậy ta định nghĩa một hệ cơ số biến đổi $m-n$: chữ số thứ nhất có cơ số m , chữ số thứ hai có cơ số $m - 1$, ..., chữ số thứ n có cơ số $m - n + 1$.

Mọi số x trong khoảng $[0, P(m, n) - 1]$ có thể biểu diễn duy nhất bằng dãy chữ số tương ứng. Để ánh xạ một chỉnh hợp sang số, duy trì danh sách

$$L = [1, 2, \dots, m]$$

đánh chỉ số từ 0. Với chỉnh hợp $p_1, p_2, \dots, p_n$, lần lượt tìm chỉ số của p_i trong L , ghi lại chỉ số đó rồi xóa p_i khỏi L .

Ví dụ, chọn 2, 4, 3 từ 1, 2, 3, 4, 5. Ban đầu $L = [1, 2, 3, 4, 5]$; lấy 2 có chỉ số 1, rồi $L = [1, 3, 4, 5]$; lấy 4 có chỉ số 2, rồi $L = [1, 3, 5]$; lấy 3 có chỉ số 1. Mã là 121 trong hệ cơ số 5, 4, 3, ứng với

$$1 \cdot 4 \cdot 3 + 2 \cdot 3 + 1 = 19.$$

Ngược lại, từ số 19 có thể khôi phục mã 121 và chỉnh hợp 243.

5 Kết luận

Bài viết đã giới thiệu và phân tích một số hàm băm thường dùng, đồng thời mở rộng sang quá trình xây dựng song ánh giữa cấu trúc tổ hợp và số tự nhiên. Băm là một cấu trúc dữ liệu hiệu quả, thể hiện rõ tư tưởng dùng không gian đổi lấy thời gian. Khi giới hạn bộ nhớ trong các kỳ thi ngày càng thoáng hơn, biết tận dụng bộ nhớ để đổi lấy thời gian là một kỹ năng quan trọng; băm có thể giúp rất nhiều trong việc đó. Cần căn cứ vào đặc điểm của đề để chọn cấu trúc dữ liệu và hàm băm phù hợp.

Tác giả cũng nêu rằng với tổ hợp chưa tìm được một cách song ánh tốt như với hoán vị, và mong được thảo luận thêm.

Tài liệu tham khảo

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*, Second Edition, MIT Press, 2001.
- Liu Rujia, Huang Liang, *Nghệ thuật thuật toán và thi tin học*, Nhà xuất bản Đại học Thanh Hoa, 2004.
- Lu Kaicheng, Lu Huaming, *Toán tổ hợp*, ấn bản thứ ba, Nhà xuất bản Đại học Thanh Hoa, 2002.

A Ghi chú về phụ lục mã nguồn

Bản gốc kèm mã C của nhiều hàm băm chuỗi thường dùng: RS, JS, P. J. Weinberger, ELF, BKDR, SDBM, DJB và AP. Vì đây là các đoạn mã mẫu và yêu cầu hiện tại không cần dịch code templates, phụ lục này chỉ ghi nhận danh sách hàm. Hai hàm JS và SDBM đã được trích trong thân bài vì chúng là ví dụ trực tiếp được tác giả khuyến nghị.